

```

[root@hemant sdt]# ./test
Enter a number between 1 and 10 : 3
you entered 3
in fun
[root@hemant sdt]# ./test
Enter a number between 1 and 10 : 7
you entered 7
in foo
[root@hemant sdt]# ./test
Enter a number between 1 and 10 : 5

[hemant@hemant-fedora ~]$ stap -v script.stp
Pass 1: parsed user script and 106 library script(s) using 218616virt/35848es/3072shr/32
Pass 2: analyzed script: 3 probe(s), 8 function(s), 8 event(s), 8 global(s) using 218616
ms.
Pass 3: translated to C into "/tmp/stap00J8Ed/stap_740942909a7c4bb085f1f4947d01c785_1494_
in 38usr/208sys/2655real ms
Pass 4: compiled C into "stap_740942909a7c4bb085f1f4947d01c785_1494.ko" in 1600usr/170sys
ms.
Pass 5: starting run.
input marker hit and value = 3
Marker <less> hit, call to fun()
input marker hit and value = 7
Marker <greater> hit, call to foo()
input marker hit and value = 5
Marker <less> hit, call to fun()

```

Figure 3: test runs three times with three different values, after which, see the output

[15]	.rodata	PROGBITS	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[16]	.stap.sdt.base	PROGBITS	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[17]	.eh_frame_hdr	PROGBITS	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[18]	.eh_frame	PROGBITS	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[19]	.init_array	INIT_ARRAY	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[20]	.fini_array	FINI_ARRAY	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[21]	.jcr	PROGBITS	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[22]	.dynamic	DYNAMIC	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[23]	.got	PROGBITS	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[24]	.got.plt	PROGBITS	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[25]	.data	PROGBITS	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[26]	.bss	NOBITS	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[27]	.comment	PROGBITS	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[28]	.note.stap.sdt	NOTE	0000000000000000	0000000000000000	0000000000000000	0000000000000000
[29]	.shstrtab	STRTAB	0000000000000000	0000000000000000	0000000000000000	0000000000000000

Figure 4: readelf output showing the .stap.sdt.base and the .note.stap.sdt sections (ALOC bit is turned off for .note.stap.sdt)

Let's get some more information at a particular point in the program about some important variables. That's where the arguments come to the fore. Here is a simple program showing their usage:

```

#include <stdio.h>
#include <sys/sdt.h>

int main(void)
{
    int no;
    printf(
        scanf("%d", &no);

    STAP_PROBE1(test, input, no);
    if (no > 5) {

```

```

        STAP_PROBE(test, greater);
        foo();
    } else {
        STAP_PROBE(test, lesser);
        fun();
    }
}

```

In the above code snippet, we have used three markers. Again, the program isn't important but the usage of the marker is. See the first one: **STAP_PROBE1**. Notice '1' as the suffix? This specifies that you are sending one argument, which in this case is *no*. Similarly, if we use two arguments, then the suffix would be '2'. It can go on till 12. After that, you have to specify N for N number of arguments. For more information on this, skim through <sys/sdt.h>.

Now, compile the program. This time, let's write a script for Systemtap to be executed in a file (not in the command line). Systemtap scripts should end with .stp. Here is the script:

```

probe process("/home/hemant/backup/home_backup/prac/
test").mark("input") {
    printf("input marker hit and value = %d\n",
        $arg1);
}
probe process("/home/hemant/backup/home_backup/prac/
test").mark("greater") {
    printf("Marker <greater> hit, call to foo()\n");
}
probe process("/home/hemant/backup/home_backup/prac/
test").mark("lesser") {
    printf("Marker <less> hit, call to fun()\n");
}

```

Systemtap uses C-style statements like *printf()*. The first probe point is important. We are asking Systemtap to print the value *\$arg1*. If we had to use more arguments, then we would have used *\$arg2*, *\$arg3*, etc, to retrieve those values. To run this script, use:

```
$ stap -v script.stp
```

You can refer to Figure 3. Remember to run the program in another terminal. We can also send a string pointer as an argument and print that in the script. You are free to try that. Here's a look at what happens behind the scenes.

Location of SDT markers in the ELF

The standard format for object code in Linux is Executable and Linkable Format (ELF). We know that an ELF consists of sections (like the .data, .rodata, .text, etc). When we compile a program containing SDT markers, two sections